Topic: Terraform Meta-Arguments — count, for_each, depends_on, provider, lifecycle + Version Constraints & state rm / mv

(Your notes, cleaned up from dictation + screenshots — Lecture 10)

Quick recall

> **Definition (lecture slide):** These are some special constructs in Terraform which are available for **Resource and Module blocks**. Meta-arguments help us achieve certain requirements within the resource block. They come in handy in situations like creating resources in the same cloud provider but **in different regions**, or when we have to **declare dependencies in places where Terraform is not able to identify the dependency itself**. Meta-arguments let you **customize the behaviour of modules and resources** in your code.
>
> **Definition (reference):** Terraform meta-arguments are **special, built-in configuration arguments that control how HashiCorp Terraform manages, scales and deploys infrastructure resources**. Standard arguments define the properties of a resource (like a server's size or name). Meta-arguments change **how Terraform interacts with the resource block itself**. They're universally supported across all providers (AWS, GCP, Azure).

In one line: normal arguments describe *the resource*. Meta-arguments describe *how Terraform should handle the resource*. They modify its behaviour.

The 5 meta-arguments

#	Meta-argument	One-liner	Works on
1	`count`	Create **N near-identical copies**	resource, module, data
2	`for_each`	**Loop over a map or set** and create one copy per key	resource, module, data
3	`depends_on`	**Wait until another resource is ready** (explicit dependency)	resource, module, data
4	`provider`	Use a **non-default provider configuration** (another region/account)	resource, data (modules use `providers = {}`)
5	`lifecycle`	Override the standard **create / update / destroy** behaviour	resource only

---

1. `count`: create several copies of the same resource

- **Purpose:** creates a **specific number of near-identical infrastructure objects** from a single block.
- **Behaviour:** accepts a **whole number**.
- **Tracking:** uses a **zero-indexed** tracking system, `count.index`, to distinguish between instances.
- **Limitation:** adding or removing items **from the middle** of a list indexed by `count` can force Terraform to destroy and recreate unrelated resources.

By default every resource block creates **one** object. When you want more than one, use `count`:

```
resource "aws_instance" "linux" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  count         = 2                              # this makes 2 EC2 instances
  tags = {
    Name = "Demo-server-${count.index}"          # Demo-server-0, Demo-server-1
  }
}
```

Every instance should have a **different name**, so we tell the two apart in the tags with `${count.index}`.

```
[root@Devops-server proejct1]# terraform apply --auto-approve
Plan: 2 to add, 0 to change, 0 to destroy.
aws_instance.linux[0]: Creating...
aws_instance.linux[1]: Creating...
```

Console: **Demo-server-0** (`i-0232abcfdc2f2e622`) and **Demo-server-1** (`i-01c5bdbf96b837d89`), both t2.micro in ap-south-1a.

> 📝 The addresses are **numbers**: `aws_instance.linux[0]` , `[1]` . If you later remove the *first* item of a list driven by `count` , every item after it shifts one position, and Terraform destroys and recreates them. That's the limitation `for_each` fixes.

2. `for_each` : loop over a map or set

- **Purpose:** create multiple resource instances **mapped to specific keys**, to avoid the tracking issues of `count` .
- "*Loop over a list or map.*" (In practice it takes a **map** or a **set of strings**. A list must be converted with `toset()` .)

Use case: we want different settings per item, e.g. dev on t2.micro and prod on t2.nano. In the lab, it's a **web** server and a **db** server with different instance types:

```
resource "aws_instance" "linux" {
  ami      = "ami-0cca134ec43cf708f"
  for_each = {
    web = "t2.micro"
    db  = "t2.medium"
  }
  instance_type = each.value                    # t2.micro / t2.medium
  key_name      = "ram_aws"
  tags = {
    Name = "Ec2-server-${each.key}"              # Ec2-server-web / Ec2-server-db
  }
}
```

Expression	Meaning	Here
`each.key`	the map key (or the set element)	`web` , `db`
`each.value`	the map value (same as key for a set)	`t2.micro` , `t2.medium`

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_instance.linux["web"]: Creating...
aws_instance.linux["db"]: Creating...
```

Console: **Ec2-server-db** (t2.medium, `i-039ec58a26439b965`) and **Ec2-server-web** (t2.micro, `i-0aa0707393cd0a499`).

Why `for_each` is safer than `count`: the addresses are **names**, `linux["web"]` and `linux["db"]`, not positions. Removing `web` destroys only `web`. `db` is untouched.

> ⚠️ `count` and `for_each` **can't be used together** in the same block. The `for_each` keys must be known at plan time (not values that are only "known after apply").

`count` vs `for_each`

	`count`	`for_each`
Input	number	map or set(string)
Address	`res[0]`, `res[1]`	`res["web"]`, `res["db"]`
Per-item differences	only via `count.index` lookups	natural: `each.key` / `each.value`
Remove an item in the middle	later items **shift** → **recreated**	only that item is destroyed
Best for	N identical copies, or `count = var.create ? 1 : 0` (on/off switch)	named things with their own settings

3. `depends_on`: wait until another resource is ready

- **Purpose:** enforces an **explicit deployment order** when Terraform can't naturally discover the dependency.
- **Behaviour:** accepts a **list** of resources or modules.
- **Usage:** best used as a **last resort**. Overusing it overcomplicates the dependency graph and slows down parallel deployments.

Meaning: one thing depends on the existence of another. Example: **my front-end server shouldn't come up until my DB server is up.**

Lab: create the EC2 instance **only if the S3 bucket deployed successfully**.

```
# Step 1. create s3 bucket first.
resource "aws_s3_bucket" "first" {
  bucket = "demobucket"
  tags = {
    Name = "my_buck"
  }
}
# Step 2. create ec2 instance only if s3 resources successfully deployed
resource "aws_instance" "redhat" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  depends_on    = [aws_s3_bucket.first]
  key_name      = "ram_aws"
  tags = {
    Name   = "Second_server"
    source = "terraform"
  }
}
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_s3_bucket.first: Creating...

Error: creating S3 Bucket (demobucket): operation error S3: CreateBucket, https response err
StatusCode: 409, ... BucketAlreadyExists: The requested bucket name is not available. The bu
namespace is shared by all users of the system. Please select a different name and try again
```

What happened: the bucket name wasn't unique (S3 names are **global** across all AWS accounts), so the bucket wasn't created. **Because of `depends_on`, the EC2 instance wasn't created either.** Without `depends_on`, the two have no link, so Terraform would create them **in parallel** and the EC2 would have been created even though the bucket failed.

> 📝 **Implicit vs explicit dependencies.** If the EC2 *referenced* the bucket (e.g. `user_data = aws_s3_bucket.first.bucket`), Terraform would already know the order (Chapter 02). That's an **implicit** dependency. `depends_on` is for **hidden** dependencies Terraform can't see from the code, e.g. an app that reads a bucket at boot, or an IAM policy that must exist before a Lambda runs.

4. `provider`: same cloud, different region (or account)

- **Purpose:** assigns a **non-default provider configuration** or region to a specific block.
- **Behaviour:** decides which cloud account or region manages the resource.
- **Usage:** ideal for **multi-region** networking or **cross-account** components.

So far, every EC2 instance was created in one region (Mumbai). Can we create two instances, **one in Mumbai and one in North Virginia**, from the same code? By default, no. One `provider "aws"` block = one region. The solution is **multiple provider blocks with an `alias`** (a shortcut / short name, just like a shell alias), plus the `provider` meta-argument on each resource:

```
provider "aws" {
  region = "ap-south-1"
  alias  = "mumbai"
}

provider "aws" {
  region = "us-east-1"
  alias  = "NV"
}

resource "aws_instance" "linux" {
  ami           = "ami-0cca134ec43cf708f"        # Mumbai AMI
  instance_type = "t2.micro"
  key_name      = "ram_aws"                      # Mumbai key pair
  provider      = aws.mumbai
  tags = {
    Name = "Mumbai_server"
  }
}

resource "aws_instance" "linux1" {
  ami           = "ami-03a6eaae9938c858c"        # N. Virginia AMI (AMIs are per region!)
  instance_type = "t2.micro"
  key_name      = "ram_nv"                       # key pairs are per region too
  provider      = aws.NV
  tags = {
    Name = "NV_server"
  }
}
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_instance.linux1: Creation complete after 32s [id=i-047dc24d2ab5a8c35]
aws_instance.linux: Creation complete after 43s [id=i-0f46d9addbc632bf1]
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

Console: **NV_server** in **us-east-1b** and **Mumbai_server** in **ap-south-1a**.

> 📝 Notes:
>
> - The syntax is `provider = aws.<alias>`, with **no quotes**.
> - The **AMI ID and key pair must exist in that region**. The same AMI ID doesn't work in another region.

- A provider block **without** an alias is the **default**, used by every resource that doesn't set `provider`. If *all* AWS provider blocks have aliases, such resources get an unconfigured AWS provider and fail (no region). Keep one default and alias the extras.
- For **modules**, pass providers with `providers = { aws = aws.NV }`.
- **Cross-account:** add `assume_role { role_arn = "arn:aws:iam::<acct>:role/..." }` to an aliased provider.

5. `lifecycle`: override create / update / destroy behaviour

- **Purpose:** a **nested block** used to override Terraform's standard create, update or destroy behaviour.

Sub-argument	What it does
`create_before_destroy`	Creates the **replacement before deleting** the old object, to avoid downtime
`prevent_destroy`	**Rejects any plan** that would destroy this resource (e.g. production databases)
`ignore_changes`	**Ignores changes** made outside Terraform (or in code) to the listed attributes
`replace_triggered_by`	Forces **recreation** of this resource when another resource/attribute changes
`precondition` / `postcondition`	Custom checks with a clear error message (Terraform ≥ 1.2)

Lab: `prevent_destroy`

Start with a simple EC2 instance, then add the lifecycle block:

```
resource "aws_instance" "linux1" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = {
    Name = "Web_server"
  }
  lifecycle {
    prevent_destroy = true   ##### to prevent destroy
  }
}
```

(The first draft was missing the `}` that closes `lifecycle`. Every block needs its own closing brace.)

```
Plan: 1 to add, 0 to change, 0 to destroy.
aws_instance.linux1: Creation complete after 33s [id=i-07a3ec976057889b7]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
[root@Devops-server proejct1]# terraform destroy --auto-approve
```

The danger: we often have the habit of running `terraform destroy --auto-approve`. With auto-approve it **doesn't ask any question**. It just deletes. If we didn't want that instance deleted, we're now in a panic: rollback, recovery, etc. With `prevent_destroy = true`, Terraform refuses:

```
Plan: 0 to add, 0 to change, 1 to destroy.

Error: Instance cannot be destroyed

  on main.tf line 1:
   1: resource "aws_instance" "linux1" {

Resource aws_instance.linux1 has lifecycle.prevent_destroy set, but the plan calls for this
resource to be destroyed. To avoid this error and continue with the plan, either disable
lifecycle.prevent_destroy or reduce the scope of the plan using the -target option.
```

To really delete it, set `prevent_destroy = false` and run destroy again.

> ⚠️ `prevent_destroy` only protects while the **resource block is in the code**. If someone deletes the whole block (or the module call), Terraform destroys it without complaint. It also blocks **replacements** (e.g. an AMI change that forces a new instance). For real protection on databases, also enable the provider-side flag, e.g. `deletion_protection = true` on RDS or `disable_api_termination = true` on EC2.

Lab: `ignore_changes`

Sometimes things change and we **don't want Terraform to act on those changes**. A typical case is **tags**: they change all the time (cost tools, other teams, the console), and we don't want every plan trying to put them back.

```
  lifecycle {
    prevent_destroy = true
    ignore_changes  = [tags, instance_type]
  }
```

```
[root@Devops-server proejct1]# terraform apply --auto-approve
aws_instance.linux1: Refreshing state... [id=i-07a3ec976057889b7]

No changes. Your infrastructure matches the configuration.
```

Even though `tags` / `instance_type` differ between the code and AWS, Terraform reports **No changes**, because those attributes are ignored.

> 📝 `ignore_changes = all` ignores every attribute (rarely a good idea). Ignoring `instance_type` means even **your own code change** to it is ignored. Use it only for attributes managed elsewhere (autoscaling, a console team, an external tool).

The other two, in short

```
resource "aws_instance" "web" {
  # ...
  lifecycle {
    create_before_destroy = true                          # new one up, then old one removed
    replace_triggered_by  = [aws_launch_template.web.latest_version]
    precondition {
      condition     = startswith(var.ami, "ami-")
      error_message = "AMI must start with ami-."
    }
  }
}
```

(`create_before_destroy` needs names that can exist twice at the same time. A fixed `name` on, for example, a security group will clash, so use `name_prefix`.)

---

6. Terraform & provider version constraints *(very important interview point)*

HashiCorp actively develops Terraform and its providers. **Every new version brings new features or changed behaviour.** If you don't fix the version, the same code can behave differently, or break, on another machine. Set `required_version` (Terraform CLI) and `required_providers.version` (providers) to make updates **predictable**.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.52.0"
    }
    random = {
      version = "~> 3.6.2"
    }
  }

  required_version = "~> 1.1.9"
}
```

Version constraint operators

Required version	Meaning	Considerations
`1.7.5` (or `= 1.7.5`)	Only **exactly** v1.7.5	To upgrade, first edit the constraint
`>= 1.7.5`	v1.7.5 **or greater**	Includes v2.0.0 and above (risky)
`~> 1.7.5`	Any **v1.7.x ≥ 1.7.5**, but **not v1.8** or later	Patch updates should be non-disruptive
`>= 1.7.5, < 1.9.5`	v1.7.5 or greater, but **less than** v1.9.5	Avoids specific version updates
`!= 1.8.0`	Anything except that version	Skip a known-bad release

`~>` (the pessimistic operator) allows only the rightmost number to increase:

- `~> 3.74.0` → `>= 3.74.0, < 3.75.0` (patches only)
- `~> 3.74` → `>= 3.74, < 4.0` (minors + patches)
- `~> 5.0` → any 5.x

Lab: directory A (pinned) vs directory B (unpinned)

On the Terraform Registry, the **latest AWS provider was v6.52.0** (494 versions available). We choose an **old one, 3.74.0** (published Jan 28, 2022), and copy its `aws_s3_bucket` example.

```
[root@Devops-server ~]# mkdir a b
[root@Devops-server ~]# cd a/
[root@Devops-server a]# vim s3.tf
[root@Devops-server a]# vim provider.tf
```

```
# a/provider.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~>3.74.0"
    }
  }
}
# a/s3.tf  (example from the 3.74.0 docs)
resource "aws_s3_bucket" "b" {
  bucket = "my-tf-test-bucket"
  acl    = "private"
  tags = {
    Name        = "My bucket"
    Environment = "Dev"
  }
}
```

Because we work on a **lower version**, we must **explicitly define it**. `~>3.74.0` means **3.74.x only: at least 3.74.0 and below 3.75.0**.

```
[root@Devops-server a]# terraform init
- Finding hashicorp/aws versions matching "~> 3.74.0"...
- Installing hashicorp/aws v3.74.3...
- Installed hashicorp/aws v3.74.3 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider selections it m
```

It installed **3.74.3** (the newest patch inside `~> 3.74.0`), **not** the latest 6.52.0. The selection is **locked** in `.terraform.lock.hcl`:

```
provider "registry.terraform.io/hashicorp/aws" {
  version     = "3.74.3"
  constraints = "~> 3.74.0"
  hashes = [
    "h1:h4TYqgRKTuuWfZtxJnEGcs/NxGCaxZ4jr0IwTfgZDRM=",
    "zh:25401cd4667d0496caf7e92e74ecef7c98cf74465570705cda2207770c27ff6c",
    ...
  ]
}
```

Directory B: someone who doesn't follow the rule. B asks A: "*I want to create the same thing, can you give me your files?*" A gives all the files. **B removes** `provider.tf` and runs init:

```
[root@Devops-server b]# cp /root/a/* .
[root@Devops-server b]# rm -rf provider.tf
[root@Devops-server b]# terraform init
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.52.0...
- Installed hashicorp/aws v6.52.0 (signed by HashiCorp)

[root@Devops-server b]# terraform validate
Warning: Argument is deprecated
  with aws_s3_bucket.b,
  on s3.tf line 3, in resource "aws_s3_bucket" "b":
   3:   acl    = "private"
acl is deprecated. Use the aws_s3_bucket_acl resource instead.

Success! The configuration is valid, but there were some validation warnings as shown above.
```

Why? In 3.x, `acl` could be set as an **argument** of `aws_s3_bucket`. In the newer provider (4.0 onwards, and 6.52 here) it's a **separate resource**, `aws_s3_bucket_acl`. The code was written for one version and run with another, and **that mismatch creates chaos in production.**

> **Rule:** always write Terraform code **with the version**, and state which version it works with, so everyone knows what the code was built for and which arguments are allowed.

Fixing B: B realizes the mistake, copies `provider.tf` back from A, and runs `init` again:

```
[root@Devops-server b]# terraform init
- Reusing previous version of hashicorp/aws from the dependency lock file

Error: Failed to query available provider packages

Could not retrieve the list of available versions for provider hashicorp/aws: locked provide
registry.terraform.io/hashicorp/aws 6.52.0 does not match configured version constraint ~> 3
must use terraform init -upgrade to allow selection of new versions
```

B's **lock file already holds 6.52.0**, which conflicts with the new constraint. Plain `init` respects the lock file, so it fails. To re-select versions and rewrite the lock:

```
[root@Devops-server b]# terraform init --upgrade
- Finding hashicorp/aws versions matching "~> 3.74.0"...
[root@Devops-server b]# terraform validate
Success! The configuration is valid.
```

*(`-upgrade` here actually **downgrades** to 3.74.3. The flag means "ignore the lock file and pick again within the constraints".)*

> 📝 **Commit `.terraform.lock.hcl` to Git.** That's what makes every engineer and the CI pipeline install **exactly** the same provider build. Don't commit `.terraform/` . **Modern S3 code** (provider ≥ 4): `aws_s3_bucket` + `aws_s3_bucket_ownership_controls` + `aws_s3_bucket_acl` (ACLs are disabled by default on new buckets since April 2023, so usually just skip ACLs).

---

7. `terraform state rm` and `terraform state mv` *(interview question)*

Setup: one EC2 instance and one S3 bucket, both managed by Terraform:

```
########### Launch Ec2 Instance in Mumbai region
resource "aws_instance" "linux" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = { Name = "Test_server", source = "terraform" }
}
########### Launch S3 Bucket in Mumbai region
resource "aws_s3_bucket" "first" {
  bucket = "linux23212google123"
  tags   = { Name = "my_buck" }
}
```

`terraform state rm`: stop managing, without destroying

> The `terraform state rm` command **removes a resource from your Terraform state file without destroying the actual infrastructure**. Once removed, Terraform **stops tracking** the object, leaving it fully intact in your cloud provider.

Use it to **release a resource from Terraform's control**:

```
[root@Devops-server project1]# terraform state list
aws_instance.linux
aws_s3_bucket.first
[root@Devops-server project1]# terraform state rm aws_s3_bucket.first
Removed aws_s3_bucket.first
Successfully removed 1 resource instance(s).
[root@Devops-server project1]# terraform state list
aws_instance.linux
```

The bucket still exists in AWS. Terraform just doesn't know about it any more.

> ⚠️ **Also remove the block from the code.** If `resource "aws_s3_bucket" "first"` stays in `main.tf`, the next `plan` wants to **create it again** (and fails, because the name is taken). State rm = "forget"; it's the opposite of `terraform import` (Chapter 03). **Modern alternative (Terraform ≥ 1.7):** a `removed {}` block that reviews like code:
>
> ```
> removed {
>   from = aws_s3_bucket.first
>   lifecycle { destroy = false }
> }
> ```

`terraform state mv`: rename / relocate, without recreating

> The `terraform state mv` command **renames or relocates resources within your Terraform state file without destroying or recreating the actual infrastructure**. It updates the mapping between your updated code and the existing cloud assets.

Use it when you **want to change the name you gave a resource** in the code:

```
[root@Devops-server project1]# terraform state mv aws_s3_bucket.first aws_s3_bucket.hello
```

Rename the block in the code too (`resource "aws_s3_bucket" "hello"`). Otherwise the plan sees `hello` in state but `first` in code, and wants to destroy one and create the other.

Why it matters: if you just rename the block in code **without** `state mv`, Terraform thinks `first` was deleted and `hello` is new, so it **destroys the real bucket and creates a new one**. `state mv` only rewrites the name in state, so the plan shows **No changes**.

Other uses: moving a resource **into a module** (`terraform state mv aws_instance.linux module.web.aws_instance.linux`), and `count` → `for_each` conversions (`'aws_instance.linux[0]'` → `'aws_instance.linux["web"]'`).

> **Modern alternative (Terraform ≥ 1.1):** a `moved {}` block in code (see Chapter 09). It's reviewed in a PR and applied by `plan` / `apply` for everyone, instead of a one-off command on one laptop.

Command	State	Real infrastructure	Typical use
`terraform state rm <addr>`	removed	**untouched**	Hand a resource to another team or stack, stop managing it
`terraform state mv <old> <new>`	renamed / moved	**untouched**	Rename a resource, move it into a module, `count` → `for_each`
`terraform import <addr> <id>`	added	untouched	Start managing an existing resource
`terraform destroy -target=<addr>`	removed	**deleted**	Really delete one resource

Command reference (this chapter)

```
terraform apply --auto-approve                    # count/for_each/provider labs
terraform destroy --auto-approve                  # blocked by lifecycle.prevent_destroy
terraform state list                              # linux[0], linux[1] / linux["web"], linux
terraform state show 'aws_instance.linux["web"]'  # quote addresses with [ ] or " "
terraform init                                    # installs versions matching required_prov
terraform init -upgrade                           # ignore the lock file, re-select within t
terraform providers                               # which modules require which provider ver
terraform version                                 # CLI + provider versions in use
terraform validate                                # shows deprecation warnings (acl on aws_s
terraform state rm aws_s3_bucket.first            # forget, don't destroy
terraform state mv aws_s3_bucket.first aws_s3_bucket.hello   # rename in state, don't recrea
terraform plan -target=aws_instance.linux1        # scope a plan (the prevent_destroy error
```

Still to come

- `dynamic` blocks (a "for_each inside a resource" for nested blocks like `ingress`)
- Conditional resources with `count = var.enabled ? 1 : 0`
- `for` expressions, `toset()`, `zipmap()` for building `for_each` inputs
- `create_before_destroy` with launch templates / ASGs for zero-downtime changes